

On the ARIA

Julian Lavelle

September 6, 2026

Abstract

The Additive-Recursive Inverse Approximation (ARIA) is a division method based around the idea of chunking divisors into powers of two and remainders. In doing so, we can rearrange the expression analytically and recurse to approach the true value of the divisor numerically.

In this short paper, we'll discuss the ARIA's derivation, implementation, and loss convergence. Further research is still being done on the ARIA's use cases and standings compared to other methods.

Section 1: Deriving the ARIA

Our exploration into the ARIA begins with the simple fact that $a/b = a \cdot 1/b$. With the quotient of a and b as the product of a and b 's inverse, we can shift our focus from dividing a and b to creating the inverse of b . Next, we'll note that any number can be represented as $x + y$ where $y = 2^k$, so y is the value of $b \bmod 2^k$ where 2^k is the closest lower power of two (so $k = \text{bitlen } b - 1$).

Combining these two results, we can say that:

$$\frac{a}{b} = a \cdot \frac{1}{2^k + y}$$

We can further manipulate this expression by noting that division by 2^k is simple (a bit shift on computers), so we'll write:

$$\begin{aligned} \frac{1}{2^k + y} &= \frac{2^k}{2^k(2^k + y)} = \frac{2^k + y - y}{2^k(2^k + y)} = \frac{2^k + y}{2^k(2^k + y)} - \frac{y}{2^k(2^k + y)} \\ &= \frac{1}{2^k} \left(1 - y \cdot \frac{1}{2^k + y} \right) \end{aligned}$$

Notice that $1/2^k + y = 1/b$ is exactly what we originally set out to solve. We could expand this out again:

$$= \frac{1}{2^k} \left(1 - y \cdot \frac{1}{2^k} \left(1 - y \cdot \frac{1}{2^k + y} \right) \right)$$

Or we could write an algorithmic aggregate, as introduced in my other paper, (Lavelle 2026), up to N recursions:

$$\frac{1}{y + 2^k} \approx \prod_{\lambda_{-1}=0}^{i \in [0, N]} \frac{1 - y\lambda}{2^k}$$

Here the ARIA earns two letters of its name; we begin to see the recursion, as well as the approximation due to finite N . In pseudocode, this aggregate reads:

```
x = 2^(bitlen(b) - 1)
y = b - x
lambda = 0
for i from 0 to N-1: lambda = (1 - y * lambda) >> k
```

Section 2: Loss

Because the ARIA is an approximation, it's natural that it must have some measure of loss, or error, in its result. At step i , the result is $\lambda_i = 1/2^k (1 - y\lambda_{i-1})$, so for a true value B :

$$\begin{aligned} L_i = B - \lambda_i &= B - \frac{1 - y\lambda_{i-1}}{2^k} = \frac{1 - yB}{2^k} - \frac{1 - y\lambda_{i-1}}{2^k} = \frac{-yB + y\lambda_{i-1}}{2^k} \\ &= -\frac{y}{2^k}(B - \lambda_{i-1}) = -\frac{y}{2^k} \cdot L_{i-1} \end{aligned}$$

So, loss at iteration N is then:

$$L_N = \left(-\frac{y}{2^k}\right)^N L_{-1} \implies |L_N| = \frac{y^N}{2^{kN}} |L_{-1}|$$

We know that $\lambda_{-1} = 0$ from our aggregate, so:

$$L_{-1} = B - 0 = \frac{1}{b} \implies |L_N| = \frac{y^N}{2^{kN}|b|} = \frac{1}{|b|} \left(\frac{y}{x}\right)^N$$

When the ratio of y to x small, it vanishes quickly; numbers closer to powers of two are more easily divided. When the ratio is large, this vanishes slowly. However, do note that if we allow negative y (so $x = 2^k$ uses the closest power, not just the lower power), the ratio of y to x cannot be larger than 0.5:

$$|b - x| = |y| \leq \frac{x}{2} \implies \left|\frac{y}{x}\right| \leq \frac{1}{2}$$

where that first statement is the definition of “closest;” the distance from b to $x = 2^k$ must be less than or equal to the distance from b to the next power of two in either direction. In this case we rewrite the loss as:

$$L_N = \frac{1}{|b|} \left(\left|\frac{y}{x}\right|\right)^N$$

Plugging in, we know that:

$$L_N \leq \frac{1}{|b|} \left(\frac{1}{2}\right)^N = \frac{1}{2^N |b|}$$

If we want a loss less than ℓ :

$$\frac{1}{|b|} \left(\left|\frac{y}{x}\right|\right)^N < \ell \implies \log_{|y/x|} \ell |b| < N$$

So:

$$L_N < \ell \implies N > \frac{\log_2 \ell |b|}{\log_2 (|y/x|)} = \frac{\log_2 \ell + \log_2 |b|}{\log_2 |y| - \log_2 |x|} = \frac{\text{bitlen } \ell + \text{bitlen } b}{\text{bitlen } y - k - 1}$$

where I've substituted $\text{bitlen } x = \text{bitlen } 2^k = k + 1$ as you need an extra bit for the leading 1.

Conclusions

In this paper, we've discussed the ARIA and its nature, moving through it's derivation and loss convergence alongside its simple implementation in pseudocode. Some questions are still open for research:

- How can the ARIA be optimized?
- Is the ARIA useful for big integer division?
- How does the ARIA compare to methods like Newton-Raphson?

All of which are under current research, and will be answered in the extension paper.

Lavelle, Julian. 2026. *On the Lavellian Tracker Derivative*. <https://lavelle.dev/works/On%20the%20Lavellian%20Tracker%20Derivative.pdf>.